Compiling Across the FHE/MPC Boundary

A continuation-based compiler architecture for programs whose cheapest backend changes mid-computation

Principal Investigator: Thales B. Paiva, Assistant Professor (Professor Doutor)
Departamento de Engenharia de Computação e Sistemas Digitais (PCS), Escola Politécnica, USP — LARC
`thalespaiva@larc.usp.br` • Lattes `5406900278440032`
Co-investigator: Eduardo Morais, Postdoctoral Researcher, LARC / PCS, Escola Politécnica, USP

Topic: *Compilers and development tools for FHE and MPC* • **Secondary:** *Privacy-preserving AI inference and training on encrypted or distributed data*

Artifact: `https://github.com/emmorais/baru` — open source, Apache-2.0

---

1 The architectural gap

Heir [**?**] is, by its own account and by community adoption, the de facto compiler infrastructure for homomorphic encryption: an MLIR-based stack with dialects spanning `secret`, `bgv`, `ckks`, `cggi`, library APIs for OpenFHE, Lattigo, TFHE-rs and Jaxite, and hardware targets. We take that as given. **This proposal is not an attempt to produce faster homomorphic code than Heir; inside Heir's domain that would be an engineering result, not a scientific one.**

What interests us is the boundary of that domain, which Heir's own paper draws precisely. Three consequences of a static, FHE-only lowering pipeline:

- **Nothing can be opened mid-computation.** A ciphertext cannot be inspected without the key, so declared-leakage protocol design — reveal one bit, then branch in the clear — is not expressible at any cost. (Obliviousness is *not* a difference: `convert-if-to-select` evaluates both arms, and so does our own mux. We checked, and withdrew that claim; §4.)
- **Every nonlinearity must be approximated**, and not inferred: Heir lists polynomial approximation requiring user annotation for degree and interval among its stated limitations.
- **No interactive protocol appears anywhere in the stack.** No dialect models secret sharing, garbled circuits, rounds, or a party.

None of these is a defect: they are the correct consequences of targeting FHE, where a ciphertext cannot branch and a comparison is not an arithmetic circuit. But together they fix a boundary whose cost nobody has measured, because measuring it requires a system on both sides of it.

> **Thesis.** For an important class of programs — machine-learning inference above all — the cheapest and the *most accurate* backend changes at points *inside* the program, and the switch itself is cheap. A compiler that can only lower a whole program to one paradigm must pay the worse option everywhere. Representing the switch requires an execution model in which "the rest of the computation" is a first-class object; it is awkward in a static SSA dataflow IR and natural in a continuation-based one.

Why ML is the right witness, not an arbitrary application. A convolutional network is literally an alternation of the two regimes. Linear layers are depth-shallow, massively parallel and pack into ciphertext slots — the regime FHE was built for. Nonlinearities (ReLU, max-pool, argmax) are comparisons, which FHE cannot do and must approximate, and which secret sharing computes exactly in a few rounds. The optimal program for a CNN is therefore neither an FHE nor an MPC program but an alternation, switching at every layer boundary — and the alternation is forced by the network architecture rather than contrived by us.

2 The architecture, and what is already built

Baru is a working research compiler and runtime: one Lisp-like source language over four backends — plain interpreter, 3-party MPC (ABY3-style replicated sharing, with a malicious-with-abort mode), FHE (BFV/BGV, TFHE-rs, and a CKKS arm on OpenFHE), and ZK proofs (Binius64) — with values crossing between MPC shares and FHE ciphertexts mid-computation without being exposed.

Its evaluator is a defunctionalized continuation machine: an explicit `Continuation` algebraic data type, an `Eval/Apply` task trampoline over an explicit stack, and a step budget. Two properties follow, and we distinguish sharply between what this already delivers and what it enables.

Already delivered. *(i)* A secret-dependent conditional is a continuation frame, and the interpreter's form *opens* its condition — a declared-leakage branch, which an FHE-only compiler cannot express because nothing can be opened without the key. *(ii)* Backend choice is per-operation, by dynamic dispatch through a `Backend` trait, rather than fixed by whole-program lowering — which makes a mid-program scheme switch an ordinary

operation, not an escape hatch. The flagship result rests on this: the MPC round trip a ReLU already requires *also* re-randomises the FHE ciphertext, so an arbitrarily deep network runs with **zero FHE bootstraps** on an engine supporting one multiplication.

Enabled, and proposed here (Thrust 3). Continuations are not yet serialized nor used to suspend at a round boundary — the transport blocks synchronously. Making the continuation a value turns "the rest of the protocol" into data, permitting *dynamic* round scheduling: k independent subcomputations interleaved on one party, each resumed as its message arrives. The static scheduler shows the prize — exchange count stayed at 32 for every batch size from 1 to 8 — but only for statically known dependency graphs.

3 What the boundary costs: six measured results

Full tables in Appendix A. All figures are measured elapsed times on one machine with both arms checked against the same plaintext reference.

1. The crossover is measured, and it runs both ways. In October 2026 we completed the HEIR baseline described in §4 — including the Tier A control, which confirms our own FHE arm is within 13% of HEIR-generated OpenFHE, so what follows is about paradigms and not about our implementation quality. Both arms evaluate the same MLP on the same weights with x^2 activation — the same function — at `HEStd_128_classic`, on one machine, each verified against the same plaintext reference.

Matched disclosure (both return logits)	**Heir**	**2PC LAN**	**2PC WAN**	
`64x32x10`, depth 3	167 ms	**2 ms**	434 ms	2.6× HEIR
`64x32x32x10`, depth 5	659 ms	**2 ms**	654 ms	even
`64x32x32x32x32x10`, depth 9	1493 ms	**4 ms**	**1099 ms**	1.4× ours
`64x32(×5)x10`, depth 10	**SIGSEGV**	**5 ms**	**1322 ms**	ours runs

On LAN secret sharing wins at every depth, by 101× to 377×, and the margin widens. On WAN Heir leads at depth 3 by 2.6×, the two are even at depth 5, and ours leads from there until Heir stops running. Our arm's own elapsed time moves **217**× between the two links (2 ms to 434 ms) while the homomorphic arm's does not move at all — that single fact decides the verdict, with nothing about the cryptography changed. *"Is MPC or FHE faster"* is not well-posed, and a compiler that cannot model the difference cannot make the choice.

2. Two thirds of the gap was disclosure policy, not protocol. Our arm computes the **argmax inside the protocol** and reveals only the class index; HEIR returns the whole logit vector and lets the client take the maximum in the clear. The comparison had been charging us for strictly more work *and* crediting us with strictly less disclosure. Removing the private argmax — matching HEIR's disclosure — takes wide-area exchanges from 20 to **8** at depth 3: **the private argmax was 60% of wide-area cost.** It is also a guarantee HEIR cannot offer at any price, since CKKS cannot compare. So the defensible statement is two-sided, and we give both: at matched disclosure the crossover is at depth 5; with the stronger guarantee it costs 2.5× and still leads HEIR from depth 9.

3. The boundary is a capability cliff. At multiplicative depth 10 the HEIR pipeline dies with **SIGSEGV during key generation** — at 554 MB peak resident against the 564 MB of the depth-9 run that succeeds, so *not* an out-of-memory kill, and with no "cannot find secure parameters" message either. Depth 9 completes; depth 10 does not run at all, while the secret-sharing curve continues as a straight line (+7 ms per layer on LAN). We have not isolated whether the fault lies in HEIR's parameter selection or in OpenFHE 1.5.1; both are candidates and both are being reported upstream.

4. Approximation returns the wrong answer, at depths that occur. This is the result that makes the thesis more than a performance argument. On a depth sweep of LeNet-5, **the CKKS arm returned the wrong class at 13 levels** — the reference top-two margin was 2 quanta and the worst logit error was 15. The exact (secret-sharing) arm returned the reference class at all five depths, including margins of one and two quanta. Approximate arithmetic accumulates error through every rescale and key switch; stochastic truncation's error is a carry in the last fixed-point bit, unbiased, so it does not systematically add. Since polynomial approximation of the nonlinearity is the *only* route available to an FHE-only compiler, this is a correctness consequence of the architectural boundary, not a tuning issue.

5. The same regime-dependence holds for CNNs. On LeNet-5 the margin is 32× to secret sharing on latency, ∼8× on throughput at batch 8, and 885× past the level budget (Appendix A.2–A.3). A discontinuity sits even *below* the bootstrap cliff: 13 to 15 levels forces ring $2^{15} \to 2^{16}$ and cost 61.1 → 160.0 s, 2.6× for two

levels while multiplication count moves 1.9% — security parameters, not work. A line against a staircase: that is the compiler's decision surface.

6. The language surface determines round complexity. The same CNN written in Baru as one writes a CNN in a scalar language — a loop over sliding windows, a comparison per activation — costs **507 920 messages**. Exposing already-existing batched primitives to the *language* and fusing ReLU with rescale takes the identical computation to **33 106 messages** and a private diagnosis from 794 508 ms to **911 ms** — 872×, every integer path bit-identical. The frontend, not the protocol, was the cost.

What this does not establish. The CNN margins (result 4) come from *our own* CKKS arm, 15× off Orion at 3.5% slot utilisation; only the MLP rows above are against external code. And the Lisp layer costs real time — the same network as a Baru program on the plain interpreter takes 299 ms against 268 ms for the hand-written Rust pipeline *with image and model encrypted*. Reducing it is Thrust 3's secondary objective.

4 The Heir comparison, in three tiers

The comparison must be designed so that the architectural claim is separable from implementation quality on either side. We propose three tiers, and we regard Tier A as a precondition for reporting B or C at all.

Tier A — Parity. ***Achieved for MLPs.*** The control is that Baru must be within a small constant factor of Heir-generated OpenFHE at matched security: **if our FHE arm is the weak link, then "switching to MPC helps" is an artifact of a weak baseline rather than a property of the architecture**, and nothing in §3 is attributable. We built it and it holds.

Network	Depth	Baru CKKS	Heir	
`64x32x10`	3	181 ms	167 ms	1.08× Heir
`784x32x10`	3	751 ms	178 ms	4.2× Heir
`64x32x32x10`	5	**384 ms**	659 ms	**1.7× ours**
`64x32x32x32x32x10`	9	1687 ms	1493 ms	1.13× Heir
`64x32(×5)x10`	11	**2281 ms**	**SIGSEGV**	**ours runs**

Same OpenFHE build Heir targets, same machine, same `HEStd_128_classic`, and all three arms now evaluate *one* fixture. Within **13%** at four of the five shapes Heir can run, faster at depth 5, and running at depth 11 where Heir segfaults. `784x32x10` is the one real deficit at 4.2× — the only wide-input shape, needing 68 rotations where same-depth `64x32x10` needs 26, so Heir's shift-network passes are doing something ours is not. That is a narrow, actionable target rather than a vague call to improve packing, and it is Thrust 1.

Tier A also produced the sharpest evidence for the thesis so far, one level below where we expected it. Our CKKS arm has two dense-layer evaluators: a direct one that rotates per output position, and an Orion-style baby-step/giant-step transform. BSGS spends *more* ciphertext multiplications (43 to 106) to spend *far fewer* rotations (337 to 26), and it is worth **5.4**–**8.1**×. **It is also opt-in, behind a flag.** So the same program, the same backend, the same security — and an 8× swing decided by a choice no cost model was making. *Which evaluator is cheapest is a property of the layer shape*, exactly as which *backend* is cheapest is a property of the program. The claim recurs at every level of the stack we have looked at, which is the strongest argument we have that it is structural rather than a coincidence of one comparison.

Tier B — Expressiveness. ***Measured, and it cost us a claim.*** We ran the three cases rather than citing them, and report the outcome in full because the methodology is part of the contribution. **(a) Exact comparison holds**: ReLU, max and argmax computed exactly (measured; §VI.17), against which HEIR's only route is polynomial approximation with user-annotated degree and interval — and §VI.16 measured our own CKKS arm being *refused* at depth 3 because its logit error exceeded the class margin, while the exact arm was right at every depth. **(b) "Secret-dependent control flow" is withdrawn**: Baru lowers such an `if` to a mux that evaluates both arms, exactly as `convert-if-to-select` does. Obliviousness forces the same shape on both, and we had asserted a difference that does not exist. **(c) Mid-protocol opening holds but is narrower than claimed**: the interpreter reveals the condition and says so, which HEIR cannot do at any price — but Baru's own circuit path has no `reconstruct` either, so our two backends disagree on that program.

Running Tier B also found three correctness bugs, all in secret-conditioned code paths no benchmark exercises: an **inverted mux** in the MPC VM (every secret `if` took the wrong branch); a branch result **truncated to the condition's one-bit width** (`(if (> x 0) 111 222)` returned 1); and a circuit-compiler arm that compiled **any unsupported or misspelled call to the constant zero** without a diagnostic. All three are fixed with both-polarity regression tests, and all 834 tests pass. No recorded figure moved, because no benchmark

program emits an `if` — a ReLU is `(* p (> p 0))` and an argmax is comparison bits times a product tree — and the only secret-conditioned `if` coverage in the tree exercised the FHE path. That is the honest case for Tier B existing: a feature documented in the language reference had no coverage on two of three backends, and a green suite said nothing about it.

Tier C — Crossover. Programs whose optimal assignment is mixed: show the per-region selector finds it and both pure strategies lose. §3 already establishes the crossover exists in both directions; what remains is showing a compiler can *predict* which side a given program falls on.

Two gates already enforced in code, because an unfair comparison is worse than none. *(i)* A public Heir issue reports an MNIST MLP at ∼30 s / 3 GB under OpenFHE's `HEStd_NotSet` — no security level at all — against ∼6 min / 26 GB at the secure ring dimension: a 12× factor between an insecure and a secure baseline. To be exact, and fair to Heir: that insecure run was the reporter's choice, not a Heir default — Heir's emitted C++ sets only multiplicative depth and plaintext modulus, leaving OpenFHE's default `HEStd_128_classic` in force, which we verified by reading the generated code. The gate exists because a baseline harness must *verify* the parameter it reports rather than assume it in either direction, and ours refuses to emit a latency unless the run reports `HEStd_{128,192,256}_classic`. *(ii)* Both arms must evaluate one fixture, with a test asserting the float evaluation reproduces the quantized classification.

5 Thrusts, and what the award buys

T1 — Close the two packing deficits Tier A localised: the wide-input layout that costs us 4.2× on `784x32x10`, and multi-channel slot packing so Tier A extends from MLPs to CNNs. Adversarial to our own prior numbers; we will report the shrunken figures.

T2 — Extending the Heir comparison (where the AWS credits go). The baseline now runs, and building it corrected our own estimate of what it needs: we had deferred it believing secure parameters required ∼26 GB, and the secure `784x32x10` baseline in fact peaks at 532 MB. So the credits are *not* requested for memory. Two gaps remain that a single machine cannot close. *(i)* Every wide-area figure we have, on both arms, comes from a transport that really sleeps rather than a real link — and §3 shows the network profile is worth 151×, so this is the largest uncontrolled variable in the whole comparison. Multi-region deployment (`sa-east-1` ↔ `us-east-1` ↔ `eu-west-1`) replaces it with measured intercontinental RTTs. *(ii)* Separating the depth-10 cliff into "Heir chose bad parameters," "OpenFHE has a bug," and "this regime needs bootstrapping" requires the larger instances where secure CKKS bootstrapping is tractable.

T3 — Serializable continuations and dynamic round scheduling. Make the continuation a value; schedule rounds dynamically by resuming whichever subcomputation's message arrives; measure against the static scheduler it generalises, on programs whose dependency graph is not statically known. Secondary objective: close the interpreter overhead reported in §3.

T4 — The per-region backend selector, validated against measured runs of every assignment on small programs. The question is not whether a cost model can be written but *how coarse a model suffices*, with "suffices" defined by measurement.

Deliverables. The open-source tool; a reproducible matched-threat-model benchmark suite spanning MPC and FHE, which the field lacks; two publications (compiler/systems on the architecture and backend selection; applied-crypto on switch-as-bootstrap); one doctoral thesis and two undergraduate research projects.

6 Required questions

Theoretical or applied; domain. Applied, contributing a compiler architecture and a measurement methodology. Domain: compilers and tooling for privacy-preserving computation, evaluated on private ML inference.

Applications, including for Amazon. Multi-organization collaboration on data that cannot be pooled — an AWS Clean Rooms-shaped problem, and a listed topic in this call. A cost model spanning MPC and FHE answers a provisioning question: *given this workload and this network, which privacy technology, and therefore how much compute?* §3 shows the answer inverts with the deployment network alone. Secondarily, deciding when cryptography beats a hardware enclave for a given program region. LGPD-driven demand for private analytics in Latin America is served from `sa-east-1` — exactly the wide-area shape we measure.

Assumptions and factors that could undermine the results.

- *Our FHE arm is at Heir parity on MLPs but not on CNNs* (15× off Orion there, 3.5% slot utilisation), so §VI.10's CNN margins are over our own implementation while §3's MLP results are not. T1 extends the Tier A footing to CNNs and will reduce our headline margins.
- *Single-machine provenance.* Wide-area latency is injected by a sleeping transport, not a real link — and §3 shows the profile is worth 217×. T2 replaces it: the largest uncontrolled variable we have.
- *The depth/precision sweep uses an untrained fixture*, whose tight class margins are a fixture property. The *mechanism* is supported; a quantitative accuracy claim for trained networks needs a trained fixture.

- *Threat models are matched but not identical in kind.* FHE lets the client go offline; 2PC needs it present throughout. Matching the adversary model does not remove this, and it may dominate for some deployments.
- *Risk to the thesis.* If a well-packed FHE arm wins in every regime the selector degenerates and the claim weakens to expressiveness alone; the experiment is designed so that outcome is publishable.
- *Risk to the comparison.* HEIR is a moving target; we pin a release, report its commit, and treat Tier A as a control, not a scoreboard.

Tool: license, materials, maintenance. License: **Apache-2.0** (`LICENSE`, and declared in `Cargo.toml`). Materials: a language reference, a complete experimental record including negative and withdrawn results, a cross-paradigm survey with methodology, and a performance handoff written to be read cold. Maintenance: 129 commits in three months; three contributors, one from USP; 832 tests green; CI on every push. *Discipline:* every engine *attests* what it implements, and asking for more is an error. We have exercised it: every ZKP result predating 2026-07-31 was withdrawn when the shipped verifier proved not to check constraint satisfaction, and an FHE scheme was deleted rather than caveated when our own analysis found it insecure.

7 Community, students, and budget

The PI's work is on this call's topics already. Thales Paiva's 2024–2026 record includes *Faster amortized bootstrapping using the incomplete NTT for free* (TCHES 2025) — squarely the call's *"improved practical FHE bootstrapping algorithms"* topic, and squarely the operation whose absence produces §3's depth-10 cliff — together with *Incompleteness in Number-Theoretic Transforms* (EuroS&P 2025), two further TCHES 2025 papers, and side-channel work on Kyber and HQC (TCHES 2024, PQCrypto 2025). The NTT results bear directly on BARU's own polynomial arithmetic, and the bootstrapping result on the one regime our measurements cannot currently reach. The team is not approaching FHE from the outside.

Based at LARC (PCS, Escola Politécnica da USP), which teaches an undergraduate cryptography course (*Fundamentos de Criptografia*) and supervises graduate work in applied cryptography; the award supports one doctoral student and two undergraduate research scholars (*iniciação científica*). A compiler project absorbs students unusually well — backends, evaluators, cost models and benchmark suites parallelize into defensible individual theses with measurable results — and the matched-threat-model benchmark suite is itself a community contribution whose absence caused a comparison we had to withdraw. Applied-cryptography funding concentrates in North America, Europe and Israel; USP is Latin America's leading research university, and this trains Brazilian students in a subfield where the region is under-represented.

Budget. Unrestricted funds, excluding administrative overhead. Scholarship lines use FAPESP's published national rates (DR-I R$5 790/mo, *iniciação científica* R$1 140/mo, in force from 2025-08-01), converted at R$5.03/USD as of 2026-10-21.

Doctoral scholarship, 12 months (FAPESP DR-I)	13 810
Undergraduate research scholarships, 2 × 12 months (FAPESP IC)	5 440
Doctoral continuation / DR-II differential	12 000
Dedicated high-memory measurement workstation	15 000
International conference travel, 3 person-trips incl. registration	21 000
Open-access and publication charges	6 000
Storage, local compute, contingency	6 750
Total unrestricted (USD)	**80 000**

The workstation is requested *alongside* the credits, not instead of them, and the distinction is methodological: every figure here was measured on one 16 GB laptop, and the project's discipline depends on year-over-year comparability on a machine whose state we control, which cloud instances cannot give. **AWS promotional credits** (40 000 USD) buy what a single machine cannot — multi-region latency, and the memory for secure bootstrapping (§5, T2).

References. [1] HEIR Contributors, *HEIR: A Compiler Infrastructure for Homomorphic Encryption*, arXiv:2508.11095, 2025. [2] A. Ebel, G. Garimella, B. Reagen, *Orion: A FHE Framework for Deep Learning*, ASPLOS 2025.

Appendix A — Measurement detail

Provenance. Every figure is an elapsed wall-clock time or an instrumented counter from a real run on one machine, recorded in `docs/experiments.md` alongside the command that produced it. No figure is composed from a cost model or extrapolated. Where a result has not been measured it is described as outstanding rather than estimated.

A.1 Heir vs. 2-party MPC, same function, same machine

OpenFHE 1.5.1 from source; `heir-py` 2026.8.1; every row reports `HEStd_128_classic` and reproduced the plaintext classification, so both §4 gates passed. Activation x^2 on both arms. HEIR `eval` excludes keygen; MPC `online` excludes preprocessing. MPC rows are the `dealer` *lower bound*; deployable `lowgear` totals are given below. LAN is 0 ms / 10 Gbit/s, WAN 50 ms one-way / 100 Mbit/s.

Network	Depth	Heir	2PC LAN	2PC WAN	exch	WAN, argmax
`64x32x10`	3	167 ms	2 ms	434 ms	8	1117 ms
`784x32x10`	3	178 ms	16 ms	479 ms	8	1168 ms
`64x32x32x10`	5	659 ms	2 ms	654 ms	12	1340 ms
`64x32x32x32x10`	7	1051 ms	3 ms	879 ms	16	1548 ms
`64x32x32x32x32x10`	9	1493 ms	4 ms	1099 ms	20	1772 ms
`64x32(×5)x10`	10	**SIGSEGV**	5 ms	1322 ms	24	1999 ms

The 2PC columns are at *matched disclosure* — both arms return the logit vector. The last column is our default, which computes the argmax inside the protocol and reveals only the class: a guarantee HEIR cannot offer at any price, costing ≈2.5× on WAN. Single runs, ≈1% jitter.
Measured slopes per added hidden layer: HEIR ≈442 ms, ours ≈222 ms on WAN and under 1 ms on LAN — half HEIR's, which is why the ratio moves monotonically our way with depth.

Four things this does not say. *(i)* These are dense MLPs, not the convolutional LeNet-5 of §A.2–A.3; the two sets compare only within themselves. *(ii)* `dealer` is a lower bound — against `lowgear` totals the LAN comparison is roughly a tie (264 ms vs. 167 + 92 ms), and the defensible LAN claim is the *online* one. *(iii)* The interaction asymmetry is unchanged: HEIR's client can go offline, ours cannot. *(iv)* The disclosure asymmetry is now measured rather than noted: see the last column.

Configuration alone was worth 2.4–3.8×. The optimized column enables truncation pairs (§VI.9) and bilinear operand fusion (§VI.13); the base column is how BARU's own `bench-2pc-nn` runs, which enables neither. Turning them on cuts exchanges from 137 to 36 at depth 10 and wide-area elapsed from 7695 ms to 1999 ms — no new algorithm, work already in the repository that was not reaching the comparison. The defect is instructive: it presented not as a wrong number but as a *correct* number for a configuration nobody would deploy, which is the failure mode a backend-aware cost model exists to prevent. A second finding from the same re-run: `bench-2pc-nn`'s wide-area column is `rounds` × RTT + bytes/bandwidth + compute — a cost model, not elapsed time — and the figures above replace it with wall clock from two parties over a transport that really sleeps.

A.2 Depth sweep: a line against a staircase

LeNet-5 with additional dense-plus-activation blocks (two levels each), one variable.

Levels	Exch	MiB	MPC, 50 ms WAN	Ring	CKKS	Ratio
11	32	4.82	1517 ms	2^{15}	43.8 s	29×
13	36	4.96	1683 ms	2^{15}	61.1 s	36×
15	40	5.10	1804 ms	2^{16}	160.0 s	89×
17	44	5.23	1937 ms	—	—	
19	48	5.37	2110 ms	—	—	

A.3 Precision: where approximation returns the wrong class

Margins in units of the fixed-point quantum (2^{-8}).

Levels	Ref. class	Margin	CKKS worst error	CKKS verdict	MPC
11	6	58 q	16 q	agrees	6
13	0	**2 q**	15 q	**wrong class (7)**	**0**
15	8	10 q	21 q	agrees, fortuitously	8
17	2	**1 q**	—	—	**2**
19	3	72 q	—	—	3

A.4 The frontend determines round complexity (872×)

MedMNIST pneumonia, one diagnosis, same weights, every integer path bit-identical.

Version	Plaintext, 10 img	Private (image)	Messages
Scalar Lisp	3465 ms	794 508 ms	507 920
+ batched vector primitives	322 ms	6446 ms	33 181
+ fused `vrelu`	299 ms	4326 ms	33 154
+ garbled nonlinearity	—	**911 ms**	33 106
Improvement	**11.6×**	**872×**	**15.4×**

A.5 Backend-aware transformations, each selected by a probe

Transformation	Why it is a backend-aware decision	Measured
Argmax as tournament, not fold	same comparisons, $n-1 \to \lceil \log_2 n \rceil$ depth	1.67×
Rescale by truncation pair	query-independent; crosses the online/offline line	3.67×
All-pairs argmax	spends bandwidth headroom to buy constant depth	1.47×
Open each operand once	reuse appears only at batch > 1	10.8× traffic
LeNet-5, wide-area, cumulative		**7.76 s → 1.41 s**

All-pairs argmax is the thesis in miniature: it performs 45 comparisons instead of 9, and is correct *only* because a separate measurement showed that workload had bandwidth headroom. On a bandwidth-bound link the opposite lowering wins, so both are kept and selected by cost model — a decision a single-paradigm compiler has no basis to make.

Appendix B — Capability matrix

Rows are sourced to Heir's paper (arXiv:2508.11095); none is inferred.

Capability	Heir	Baru
FHE schemes	BGV/BFV, CKKS, CGGI	BFV/BGV, CKKS (OpenFHE), TFHE-rs
Backend libraries	OpenFHE, Lattigo, TFHE-rs, Jaxite	OpenFHE; own NTT
Hardware targets	several (`pisa`, accelerators)	none
Secret sharing / MPC	not present	ABY3; malicious-with-abort
Garbled circuits	not present	yes (nonlinearity lowering)
FHE ↔ MPC switch	not present	both directions
Exact ReLU / max-pool	not expressible (no comparison)	exact
Nonlinearity route	polynomial approx., user-annotated	exact, or approx. by choice
Secret-dependent branch	`convert-if-to-select` (both arms)	executed under protocol
Secret-dependent loop bound	unsupported (static bound required)	executed under protocol
BGV/BFV bootstrapping	unsupported (no backend)	avoided via switch-as-bootstrap
Measured depth ceiling	SIGSEGV at mult. depth 10	no depth budget; linear in depth
Maturity, ecosystem, adoption	**substantially ahead**	research prototype

The last row is not a formality. Heir is production-grade infrastructure with institutional backing, multiple hardware targets and a community; Baru is a research prototype by a small group. The claim in this proposal is confined to the rows above it, and specifically to what the architecture makes *expressible* — which is why Tier A parity is a precondition rather than a result.